# Flow Control in Assembly

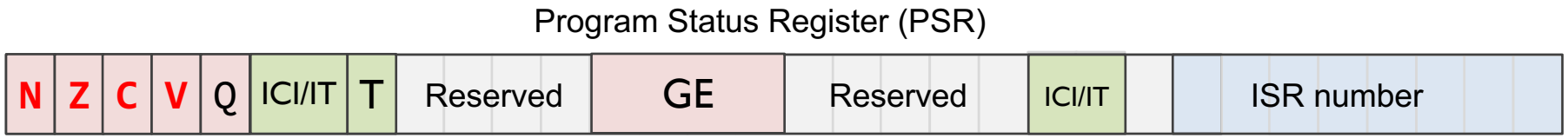
CSE 4095

Special Topics in Computer Engineering III:
Introduction to Embedded Systems
Spring 2023

Taken from Dr. Yifeng Zhu's Lecture Notes

**Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C – Chapter 6**

# Review: Condition Flags




- **Negative** bit
  - `N = 1` if most significant bit of result is 1
- **Zero** bit
  - `Z = 1` if all bits of result are `0`
- **Carry** bit
  - For unsigned addition, `C = 1` if carry takes place
  - For unsigned subtraction, `C = 0` (carry = not borrow) if borrow takes place
  - For shift/rotation, C = last bit shifted out
- **oVerflow** bit
  - `V = 1` if adding 2 same-signed numbers produces a result with the opposite sign
    - Positive + Positive = Negative, or
    - Negative + negative = Positive
  - Non-arithmetic operations does not touch V bit, such as `MOV`, `AND`, `LSL`, `MUL`

# Updating Condition Flags

- Method 1: append with "**S**"
  - `ADD r0,r1,r2` → `ADDS r0,r1,r2`
  - `SUB r0,r1,r2` → `SUBS r0,r1,r2`

- Method 2: using compare instructions

| Instruction | Brief description | Flags |
|---|---|---|
| **CMP** | Compare | N,Z,C,V |
| **CMN** | Compare Negative | N,Z,C,V |
| **TEQ** | Test Equivalence | N,Z,C |
| **TST** | Test | N,Z,C |

# Updating Condition Flags

| Instruction | Operands | Brief description | Flags |
|---|---|---|---|
| CMP | Rn, Op2 | Compare | N,Z,C,V |
| CMN | Rn, Op2 | Compare Negative | N,Z,C,V |
| TEQ | Rn, Op2 | Test Equivalence | N,Z,C |
| TST | Rn, Op2 | Test | N,Z,C |

- **Update the status flags**
  - No need to add S bit.
  - No need to specify destination register.
- Operations are:
  - `CMP` operand1 - operand2, but result not written
  - `CMN` operand1 + operand2, but result not written
  - `TST` operand1 & operand2, but result not written
  - `TEQ` operand1 ^ operand2, but result not written
- Examples:
  - `CMP r0, r1`
  - `TST r2, #5`

# Updating Condition Flags: CMP and CMN

```
CMP Rn, Operand2
CMN Rn, Operand2
```

- Update N, Z, C and V according to the result
- **CMP** subtracts Operand2 from Rn.
  - Same as SUBS, except result is discarded.
- **CMN** adds Operand2 to Rn.
  - Same as ADDS, except result is discarded.

# Example of CMP

$$f(x) = |x|$$

```
        Area absolute, CODE, READONLY
        EXPORT __main
        ENTRY

__main PROC
        CMP     r1, #0
        RSBLT   r1, r1, #0

done    B done      ; deadloop

        ENDP
        END
```

*Note: RSB = Reverse SuBtract*

# Updating Condition Flags: TST and TEQ

```
TST Rn, Operand2  ; Bitwise AND
TEQ Rn, Operand2  ; Bitwise Exclusive OR
```

- Update N and Z according to the result
- Can update C during the calculation of Operand2
- Do not affect V
- TST performs bitwise AND on Rn and Operand2.
  - Same as ANDS, except result is discarded.
- TEQ performs bitwise Exclusive OR on Rn and Operand2.
  - Same as EORS, except result is discarded.

# Unconditional Branch Instructions

| Instruction | Operands | Brief description |
|---|---|---|
| **B** | `label` | Branch |
| **BL** | `label` | Branch with Link |
| **BLX** | `Rm` | Branch indirect with Link |
| **BX** | `Rm` | Branch indirect |

- *B label*
  - cause a branch to label.
- *BL label*
  - copy the address of the next instruction into `r14` (lr, the link register), and
  - cause a branch to label.
- *BX Rm*
  - branch to the address held in Rm
- *BLX Rm*:
  - copy the address of the next instruction into `r14` (lr, the link register) and
  - branch to the address held in Rm

# Unconditional Branch Instructions: A Simple Example

```
        MOVS r1, #1
        B    target  ; Branch to target
        MOVS r2, #2  ; Not executed
        MOVS r3, #3  ; Not executed
        MOVS r4, #4  ; Not executed
target  MOVS r5, #5
```

- A label marks the location of an instruction
- Labels helps human to read the code
- In machine program, labels are converted to numeric offsets by assembler

# Condition Codes

| Suffix | Description | Flags tested |
|---|---|---|
| EQ | EQual | |
| NE | Not Equal | |
| CS/HS | Unsigned Higher or Same | |
| CC/LO | Unsigned LOwer | |
| MI | MInus (Negative) | |
| PL | PLus (Positive or Zero) | |
| VS | oVerflow Set | |
| VC | oVerflow Clear | |
| HI | Unsigned HIgher | |
| LS | Unsigned Lower or Same | |
| GE | Signed Greater or Equal | |
| LT | Signed Less Than | |
| GT | Signed Greater Than | |
| LE | Signed Less than or Equal | |
| AL | ALways | |

*Note AL is the default and does not need to be specified*

# Condition Codes

- The possible condition codes are listed below:

| Suffix | Description | Flags tested |
|---|---|---|
| EQ | EQual | Z==1 |
| NE | Not Equal | Z==0 |
| CS/HS | Unsigned Higher or Same | C==1 |
| CC/LO | Unsigned LOwer | C==0 |
| MI | MInus (Negative) | N==1 |
| PL | PLus (Positive or Zero) | N==0 |
| VS | oVerflow Set | V==1 |
| VC | oVerflow Clear | V==0 |
| HI | Unsigned HIgher | C==1 and Z==0 |
| LS | Unsigned Lower or Same | C==0 or Z==1 |
| GE | Signed Greater or Equal | N==V |
| LT | Signed Less Than | N!=V |
| GT | Signed Greater Than | Z==0 and N==V |
| LE | Signed Less than or Equal | Z==1 or N!=V |
| AL | ALways | |

*Note AL is the default and does not need to be specified*

# Signed Greater or Equal (N == V)

**`CMP r0, r1`**

perform subtraction `r0 – r1`, without saving the result

| | N = 0 | N = 1 |
|---|---|---|
| V = 0 | • No overflow, implying the result is correct.<br>• The result is non-negative,<br>• Thus `r0 – r1 ≥ 0`, i.e., `r0 ≥ r1` | • No overflow, implying the result is correct.<br>• The result is negative.<br>• Thus `r0 – r1 < 0`, i.e., `r0 < r1` |
| V = 1 | • Overflow occurs, implying the result is incorrect.<br>• The result is mistakenly reported as non-negative and in fact it should be negative.<br>• Thus `r0 – r1 < 0` in reality, i.e., `r0 < r1` | • Overflow occurs, implying the result is incorrect.<br>• The result is mistakenly reported as negative and in fact it should be non-negative.<br>• Thus `r0 – r1 ≥ 0` in reality., i.e. `r0 ≥ r1` |

Conclusions:

- If N == V, then it is signed greater or equal (GE).
- Otherwise, it is signed less than (LT)

# Signed Greater or Equal (N == V)

**CMP r0, r1**

perform subtraction r0 – r1, without saving the result

| | N = 0 | N = 1 |
|---|---|---|
| V = 0 | r0 ≥ r1 | r0 < r1 |
| V = 1 | r0 < r1 | r0 ≥ r1 |

Conclusions:

- If N == V, then it is signed greater or equal (GE).
- Otherwise, it is signed less than (LT)

# Signed Greater or Equal (N == V)

`CMP r0, r1`

perform subtraction `r0 - r1`, without saving the result

| | N = 0 | N = 1 |
|---|---|---|
| V = 0 | 1 | 0 |
| V = 1 | 0 | 1 |

Conclusions:

- If N == V, then it is signed greater or equal (GE).
- Otherwise, it is signed less than (LT)

# Signed vs. Unsigned Comparison

| Compare | Signed | Unsigned |
|---|---|---|
| > | GT | HI |
| ≥ | GE | HS |
| < | LT | LO |
| ≤ | LE | LS |
| == | EQ | |
| ≠ | NE | |



Conditional codes applied to branch instructions

| Compare | Signed | Unsigned |
|---|---|---|
| > | BGT | BHI |
| >= | BGE | BHS |
| < | BLT | BLO |
| <= | BLE | BLS |
| == | BEQ | |
| != | BNE | |

# Branch Instructions

| | Instruction | Description | Flags tested |
|---|---|---|---|
| **Unconditional Branch** | **B** *label* | Branch to label | |
| **Conditional Branch** | **BEQ** *label* | Branch if **EQ**ual | `Z = 1` |
| | **BNE** *label* | Branch if **N**ot **E**qual | `Z = 0` |
| | **BCS**/**BHS** *label* | Branch if unsigned **H**igher or **S**ame | `C = 1` |
| | **BCC**/**BLO** *label* | Branch if unsigned **LO**wer | `C = 0` |
| | **BMI** *label* | Branch if **MI**nus (Negative) | `N = 1` |
| | **BPL** *label* | Branch if **PL**us (Positive or Zero) | `N = 0` |
| | **BVS** *label* | Branch if o**V**erflow **S**et | `V = 1` |
| | **BVC** *label* | Branch if o**V**erflow **C**lear | `V = 0` |
| | **BHI** *label* | Branch if unsigned **HI**gher | `C = 1 & Z = 0` |
| | **BLS** *label* | Branch if unsigned **L**ower or **S**ame | `C = 0 or Z = 1` |
| | **BGE** *label* | Branch if signed **G**reater or **E**qual | `N = V` |
| | **BLT** *label* | Branch if signed **L**ess **T**han | `N != V` |
| | **BGT** *label* | Branch if signed **G**reater **T**han | `Z = 0 & N = V` |
| | **BLE** *label* | Branch if signed **L**ess than or **E**qual | `Z = 1 or N = !V` |

# Number Interpretation

Which is greater?

`0xFFFFFFFF` or `0x00000001`

- If they represent signed numbers, the latter is greater.
  (**`1 > -1`**).

- If they represent unsigned numbers, the former is greater.
  (**`4294967295 > 1`**).

# Which is Greater: 0xFFFFFFFF or 0x00000001?

It's **software's responsibility** to tell computer how to interpret data:

- If written in C, declare the signed *vs* unsigned variable
- If written in Assembly, use signed *vs* unsigned branch instructions

```
signed int x, y ;
x = -1;
y = 1;
if (x > y)
   ...
```

```
    MOVS r5, #0xFFFFFFFF
    MOVS r6, #0x00000001
    CMP  r5, r6
    BLE  Then_Clause
    ...
```

**BLE**: Branch if less than or equal, signed ≤

```
unsigned int x, y ;
x = 4294967295;
y = 1;
if (x > y)
   ...
```

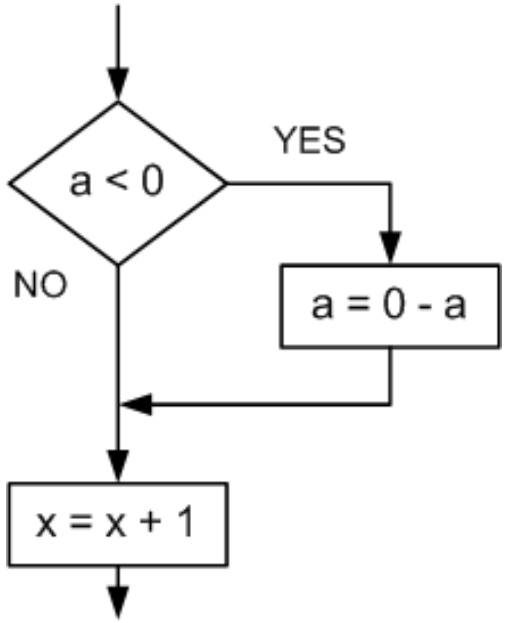
```
    MOVS r5, #0xFFFFFFFF
    MOVS r6, #0x00000001
    CMP  r5, r6
    BLS  Then_Clause
    ...
```

**BLS**: Branch if lower or same, unsigned ≤

# If-then Statement

**C Program**

```
// a is signed integer
if (a < 0 ) {
  a = 0 – a;
}
x = x + 1;
```

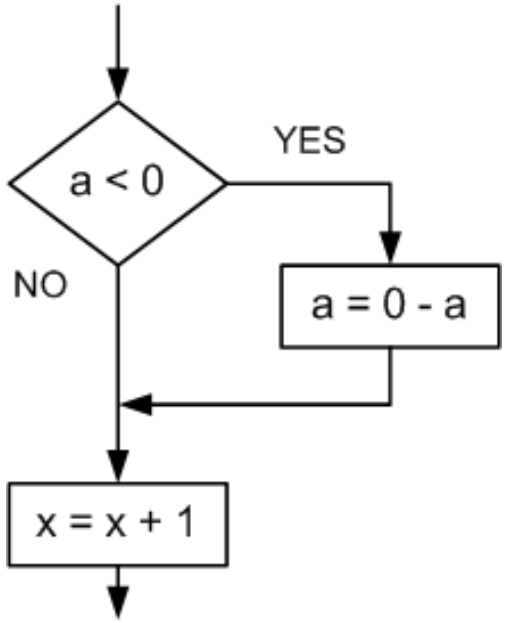



Implementation 1:

```
        ; r1 = a (signed integer), r2 = x
        CMP r1, #0          ; Compare a with 0
        BGE endif           ; Go to endif if a ≥ 0
then    RSB r1, r1, #0      ; a = - a
endif   ADD r2, r2, #1      ; x = x + 1
```

# If-then Statement

```
C Program
// a is signed integer
if (a < 0 ) {
  a = 0 – a;
}
x = x + 1;
```




Implementation 2:

```
        ; r1 = a, r2 = x
        CMP    r1, #0        ; Compare a with 0
        RSBLT  r1, r1, #0    ; a = 0 - a if a < 0
        ADD    r2, r2, #1    ; x = x + 1
```

# Compound Boolean Expression

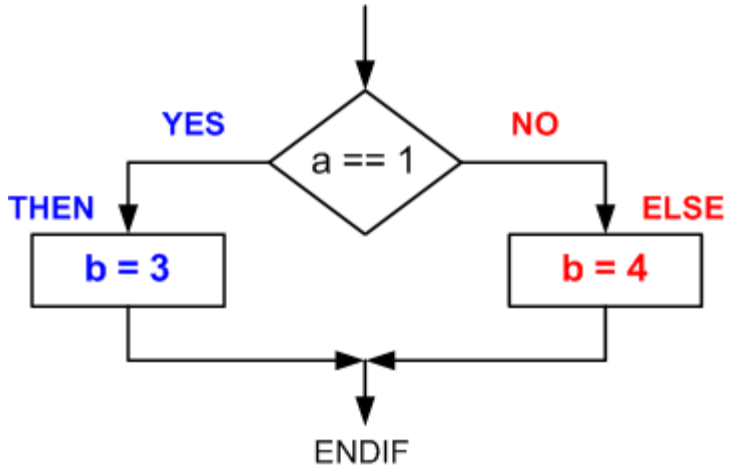
```
x > 20 && x < 25
x == 20 || x == 25
!(x == 20 || x == 25)
```

| C Program | Assembly Program |
|---|---|
| `// x is a signed integer`<br>`if(x <= 20 \|\| x >= 25){`<br>`  a = 1`<br>`}` | `  ; r0 = x`<br>`  CMP  r0, #20  ; compare x and 20`<br>`  BLE  then  ; go to then if x ≤ 20`<br>`  CMP  r0, #25  ; compare x and 25`<br>`  BLT  endif  ; go to endif if x < 25`<br>`then  MOV  r1, #1  ; a = 1`<br>`endif` |

# If-then-else

**C Program**

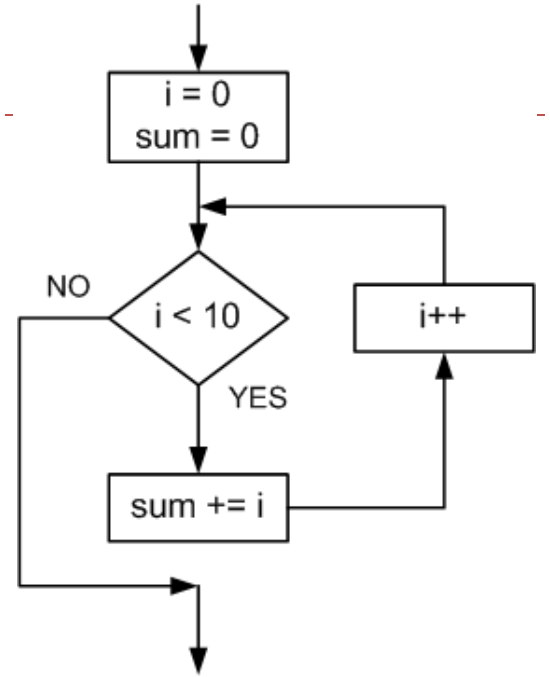
```
if (a == 1)
   b = 3;
else
   b = 4;
```




```
        ; r1 = a, r2 = b
        CMP r1, #1    ; compare a and 1
        BNE else      ; go to else if a ≠ 1
then    MOV r2, #3    ; b = 3
        B   endif     ; go to endif
else    MOV r2, #4    ; b = 4
endif
```

# For Loop

```
C Program
int i;
int sum = 0;
for(i = 0; i < 10; i++){
  sum += i;
}
```



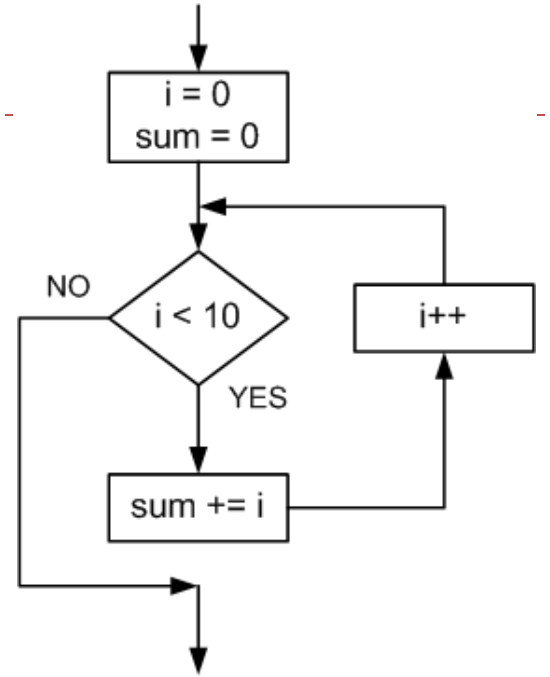

Implementation 1:

```
        MOV r0, #0   ; i
        MOV r1, #0   ; sum

        B   check
loop    ADD r1, r1, r0   ; sum += i
        ADD r0, r0, #1   ; i++
check   CMP r0, #10      ; check whether i < 10
        BLT loop         ; loop if signed less than
endloop
```

# For Loop

```
C Program
int i;
int sum = 0;
for(i = 0; i < 10; i++){
  sum += i;
}
```




Implementation 2:

```
        MOV r0, #0   ; i
        MOV r1, #0   ; sum

loop    CMP r0, #10 ; check whether i < 10
        BGE endloop ; skip if ≥
        ADD r1, r1, r0   ; sum += i
        ADD r0, r0, #1   ; i++
        B   loop
endloop
```

# Conditional Execution

| Add instruction | Condition | Flag tested |
|---|---|---|
| **ADDEQ** r3, r2, r1 | Add if EQual | Add if Z = 1 |
| **ADDNE** r3, r2, r1 | Add if Not Equal | Add if Z = 0 |
| **ADDHS** r3, r2, r1 | Add if Unsigned Higher or Same | Add if C = 1 |
| **ADDLO** r3, r2, r1 | Add if Unsigned LOwer | Add if C = 0 |
| **ADDMI** r3, r2, r1 | Add if Minus (Negative) | Add if N = 1 |
| **ADDPL** r3, r2, r1 | Add if PLus (Positive or Zero) | Add if N = 0 |
| **ADDVS** r3, r2, r1 | Add if oVerflow Set | Add if V = 1 |
| **ADDVC** r3, r2, r1 | Add if oVerflow Clear | Add if V = 0 |
| **ADDHI** r3, r2, r1 | Add if Unsigned HIgher | Add if C = 1 & Z = 0 |
| **ADDLS** r3, r2, r1 | Add if Unsigned Lower or Same | Add if C = 0 or Z = 1 |
| **ADDGE** r3, r2, r1 | Add if Signed Greater or Equal | Add if N = V |
| **ADDLT** r3, r2, r1 | Add if Signed Less Than | Add if N != V |
| **ADDGT** r3, r2, r1 | Add if Signed Greater Than | Add if Z = 0 & N = V |
| **ADDLE** r3, r2, r1 | Add if Signed Less than or Equal | Add if Z = 1 or N = !V |

# Conditional Execution

```
if (a <= 0)
  y = -1;
else
  y = 1;
```



```
a ⟶ r0
y ⟶ r1
```

```
CMP   r0, #0
MOVLE r1, #-1 ; executed if LE
MOVGT r1, #1  ; executed if GT
```

**LE**: Signed Less than or Equal
**GT**: Signed Greater Than

# Conditional Execution

```
if (a==1 || a==7 || a==11)
   y = 1;
else
   y = -1;
```



```
a ⟶ r0
y ⟶ r1
```

```
CMP   r0, #1
CMPNE r0, #7   ; executed if r0 != 1
CMPNE r0, #11  ; executed if r0 != 7
MOVEQ r1, #1
MOVNE r1, #-1
```

NE: Not Equal
EQ: Equal

# Compound Boolean Expression

| C Program | Assembly Program |
|---|---|
| `// x is a signed integer`<br>`if(x <= 20 \|\| x >= 25){`<br>`  a = 1;`<br>`}` | `; r0 = x, r1 = a`<br>`CMP  r0, #20  ; compare x and 20`<br>`MOVLE r1, #1  ; a=1 if less or equal`<br>`CMP  r0, #25  ; CMP if greater than`<br>`MOVGE r1, #1  ; a=1 if greater or equal`<br>`endif` |

# Example 1: Greatest Common Divider (GCD)

Euclid's Algorithm

```
uint32_t a, b;
while (a != b ) {
    if (a > b)
       a = a – b;
    else
       b = b – a;
}
```



```
gcd   CMP r0, r1
      SUBHI r0, r0, r1
      SUBLO r1, r1, r0
      BNE gcd
```



```
      ; suppose r0 = a and r1 = b
gcd   CMP r0, r1       ; a > b?
      BEQ end          ; if a = b, done
      BLO less         ; a < b
      SUB r0, r0, r1   ; a = a – b
      B   gcd
less  SUB r1, r1, r0   ; b = b – a
      B   gcd
end
```

# Example 2

```
// x in r0, y in r1

if ( x + y < 0 )
    x = 0;
else
    x = 1;
```

```
          ADDS r0, r0, r1
          BPL  PosOrZ
          MOV  r0, #0
          B    done
PosOrZ    MOV  r0, #1
done
```

```
        ADDS    r0, r0, r1   ;  r0 = x + y,  setting CCs
        MOVMI   r0, #0       ;  return 0 if n bit = 1
        MOVPL   r0, #1       ;  return 1 if n bit = 0
```

# Summary: Condition Codes

| Suffix | Description | Flags tested |
|---|---|---|
| EQ | EQual | Z=1 |
| NE | Not Equal | Z=0 |
| CS/HS | Unsigned Higher or Same | C=1 |
| CC/LO | Unsigned LOwer | C=0 |
| MI | MInus (Negative) | N=1 |
| PL | PLus (Positive or Zero) | N=0 |
| VS | oVerflow Set | V=1 |
| VC | oVerflow Cleared | V=0 |
| HI | Unsigned HIgher | C=1 & Z=0 |
| LS | Unsigned Lower or Same | C=0 or Z=1 |
| GE | Signed Greater or Equal | N=V |
| LT | Signed Less Than | N!=V |
| GT | Signed Greater Than | Z=0 & N=V |
| LE | Signed Less than or Equal | Z=1 or N!=V |
| AL | ALways | |

*Note AL is the default and does not need to be specified*

# Summary: Branch Instructions

| | Instruction | Description | Flags tested |
|---|---|---|---|
| **Unconditional Branch** | **B** *label* | Branch to label | |
| **Conditional Branch** | **BEQ** *label* | Branch if **EQ**ual | `Z = 1` |
| | **BNE** *label* | Branch if **N**ot **E**qual | `Z = 0` |
| | **BCS**/**BHS** *label* | Branch if unsigned **H**igher or **S**ame | `C = 1` |
| | **BCC**/**BLO** *label* | Branch if unsigned **LO**wer | `C = 0` |
| | **BMI** *label* | Branch if **MI**nus (Negative) | `N = 1` |
| | **BPL** *label* | Branch if **PL**us (Positive or Zero) | `N = 0` |
| | **BVS** *label* | Branch if o**V**erflow **S**et | `V = 1` |
| | **BVC** *label* | Branch if o**V**erflow **C**lear | `V = 0` |
| | **BHI** *label* | Branch if unsigned **HI**gher | `C = 1 & Z = 0` |
| | **BLS** *label* | Branch if unsigned **L**ower or **S**ame | `C = 0 or Z = 1` |
| | **BGE** *label* | Branch if signed **G**reater or **E**qual | `N = V` |
| | **BLT** *label* | Branch if signed **L**ess **T**han | `N != V` |
| | **BGT** *label* | Branch if signed **G**reater **T**han | `Z = 0 & N = V` |
| | **BLE** *label* | Branch if signed **L**ess than or **E**qual | `Z = 1 or N = !V` |

# Summary: Conditionally Executed

| Add instruction | Condition | Flag tested |
|---|---|---|
| ADDEQ r3, r2, r1 | Add if EQual | Add if Z = 1 |
| ADDNE r3, r2, r1 | Add if Not Equal | Add if Z = 0 |
| ADDHS r3, r2, r1 | Add if Unsigned Higher or Same | Add if C = 1 |
| ADDLO r3, r2, r1 | Add if Unsigned LOwer | Add if C = 0 |
| ADDMI r3, r2, r1 | Add if Minus (Negative) | Add if N = 1 |
| ADDPL r3, r2, r1 | Add if PLus (Positive or Zero) | Add if N = 0 |
| ADDVS r3, r2, r1 | Add if oVerflow Set | Add if V = 1 |
| ADDVC r3, r2, r1 | Add if oVerflow Clear | Add if V = 0 |
| ADDHI r3, r2, r1 | Add if Unsigned HIgher | Add if C = 1 & Z = 0 |
| ADDLS r3, r2, r1 | Add if Unsigned Lower or Same | Add if C = 0 or Z = 1 |
| ADDGE r3, r2, r1 | Add if Signed Greater or Equal | Add if N = V |
| ADDLT r3, r2, r1 | Add if Signed Less Than | Add if N != V |
| ADDGT r3, r2, r1 | Add if Signed Greater Than | Add if Z = 0 & N = V |
| ADDLE r3, r2, r1 | Add if Signed Less than or Equal | Add if Z = 1 or N = !V |

# Combination

| Instruction | Operands | Brief description |
|---|---|---|
| CBZ | Rn, label | Compare and Branch if Zero |
| CBNZ | Rn, label | Compare and Branch if Non Zero |

- Except that it does not change the status flags, `CBZ Rn,label` is equivalent to:

```
CMP Rn, #0
BEQ label
```

- Except that it does not change the status flags, `CBNZ Rn,label` is equivalent to:

```
CMP  Rn, #0
BNE  label
```

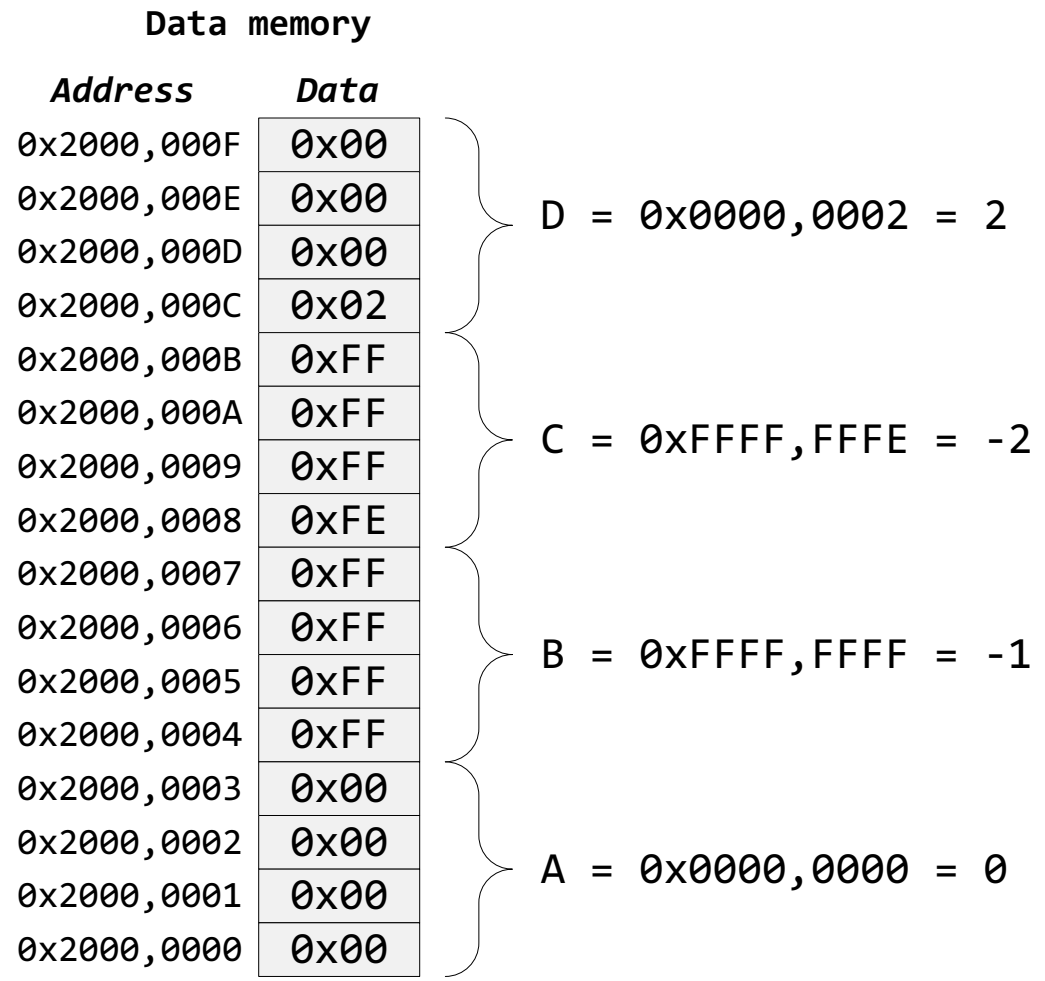
# Break and Continue

| C Program | Assembly Program |
|---|---|
| `// Find string length`<br>`char str[] = "hello";`<br>`int len = 0;`<br><br>`for( ; ; ) {`<br>`  if (*str == '\0')`<br>`  break;`<br>`  str++;`<br>`  len++;`<br>`}` | `  ; r0 = string memory address`<br>`  ; r1 = string length`<br>`  MOV  r1, #0  ; len = 0`<br><br>`loop  LDRB r2, [r0]`<br>`  CBNZ r2, notZero`<br>`  B  endloop`<br>`notZero  ADD  r0, r0, #1  ; str++`<br>`  ADD  r1, r1, #1  ; len++`<br>`  B  loop`<br>`endloop` |

# Reuse Registers

```
int A = 0;  // 0x00000000
int B = -1; // 0xFFFFFFFF
int C = -2; // 0xFFFFFFFE
int D = 2;  // 0x00000002

void main(void){
  A = B + C - D;
  return;
}
```



# Reuse Registers

```
int A = 0;  // 0x00000000
int B = -1; // 0xFFFFFFFF
int C = -2; // 0xFFFFFFFE
int D = 2;  // 0x00000002

void main(void){
  A = B + C - D;
  return;
}
```

Eight registers are used:
r0,r1,r2,r3,r4,r5,r6,r7

```
AREA myCode, CODE
   EXPORT __main
   ENTRY
__main PROC

   LDR r2, =B      ; r2 = 0x2000,0004
   LDR r3, [r2]    ; r3 = B = -1
   LDR r4, =C      ; r4 = 0x2000,0008
   LDR r5, [r4]    ; r5 = C = -2
   LDR r6, =D      ; r6 = 0x2000,000B
   LDR r7, [r6]    ; r7 = D = 2
   ADD r1, r3, r5  ; r1 = B + C
   SUB r1, r1, r7  ; r1 = B + C - D
   LDR r0, =A      ; r0 = 0x2000,0000
   STR r1, [r0]    ; Save A
   ENDP

   AREA myData, DATA
A  DCD  0
B  DCD  -1
C  DCD  -2
D  DCD  2

   END
```

# Reuse Registers

```
    AREA myCode, CODE
    EXPORT __main
    ENTRY
__main PROC

    LDR r2, =B
    LDR r3, [r2]
    LDR r4, =C
    LDR r5, [r4]
    LDR r6, =D
    LDR r7, [r6]
    ADD r1, r3, r5
    SUB r1, r1, r7
    LDR r0, =A
    STR r1, [r0]
    ENDP

    AREA myData, DATA
A   DCD  0
B   DCD  -1
C   DCD  -2
D   DCD  2

    END
```

Lifetime of r3

**8 registers used**

```
    AREA myCode, CODE
    EXPORT __main
    ENTRY
__main PROC

    LDR r2, =B
    LDR r3, [r2]
    LDR r2, =C
    LDR r5, [r2]
    LDR r2, =D
    LDR r7, [r2]
    ADD r3, r3, r5
    SUB r3, r3, r7
    LDR r2, =A
    STR r3, [r2]
    ENDP

    AREA myData, DATA
A   DCD  0
B   DCD  -1
C   DCD  -2
D   DCD  2

    END
```

Lifetime of r2

Lifetime of r2

**4 registers used**

```
    AREA myCode, CODE
    EXPORT __main
    ENTRY
__main PROC

    LDR r2, =B
    LDR r3, [r2]
    LDR r2, =C
    LDR r5, [r2]
    LDR r2, =D
    LDR r2, [r2]
    ADD r3, r3, r5
    SUB r3, r3, r2
    LDR r2, =A
    STR r3, [r2]
    ENDP

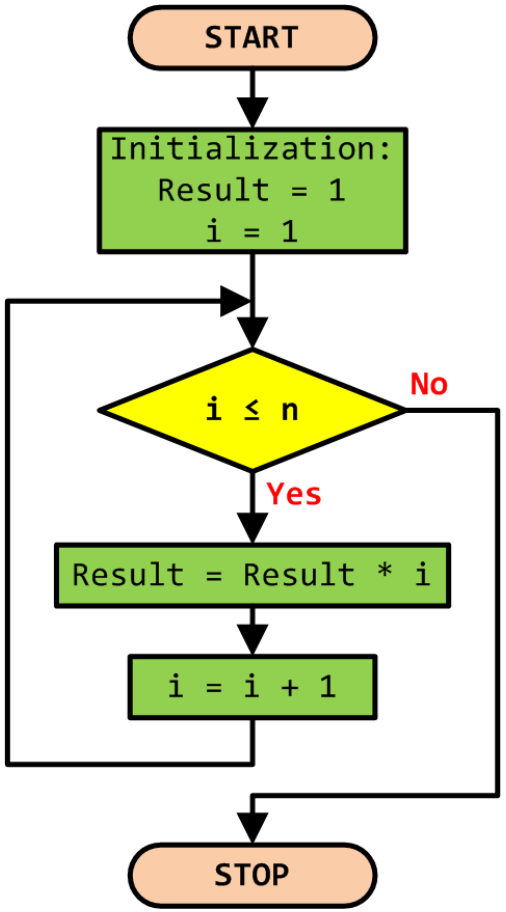
    AREA myData, DATA
A   DCD  0
B   DCD  -1
C   DCD  -2
D   DCD  2

    END
```

Reuse r2

**3 registers used**

# Example 1: Factorial Numbers




$$n! = \prod_{i=1}^{n} i = n \times (n-1) \times (n-2) \cdots \times 2 \times 1$$

# Example 1: Factorial Numbers

| C Program | Assembly Program |
|---|---|
| | `AREA factorial, CODE, READONLY`<br>`EXPORT __main`<br>`ENTRY` |
| `int main(void) {`<br>`  int result, n, i;` | `__main PROC` |
| `  result = 1;`<br>`  n = 5;` | `MOV  r0, #1  ; r0 = result`<br>`MOV  r1, #5  ; r1 = n` |
| `  for (i = 1; i <= n; i++)`<br>`    result = result * i;` | `MOV  r2, #1  ; r2 = i = 1`<br>`loop  CMP  r2, r1  ; compare i and n`<br>`BGT  stop  ; if i > n, stop`<br>`MULS r0, r2, r0  ; result *= i`<br>`ADD  r2, r2, #1  ; i++`<br>`B  loop` |
| `  while(1);`<br>`}` | `stop  B  stop`<br>`ENDP`<br>`END` |

# Example 2: Counting Ones in a Word

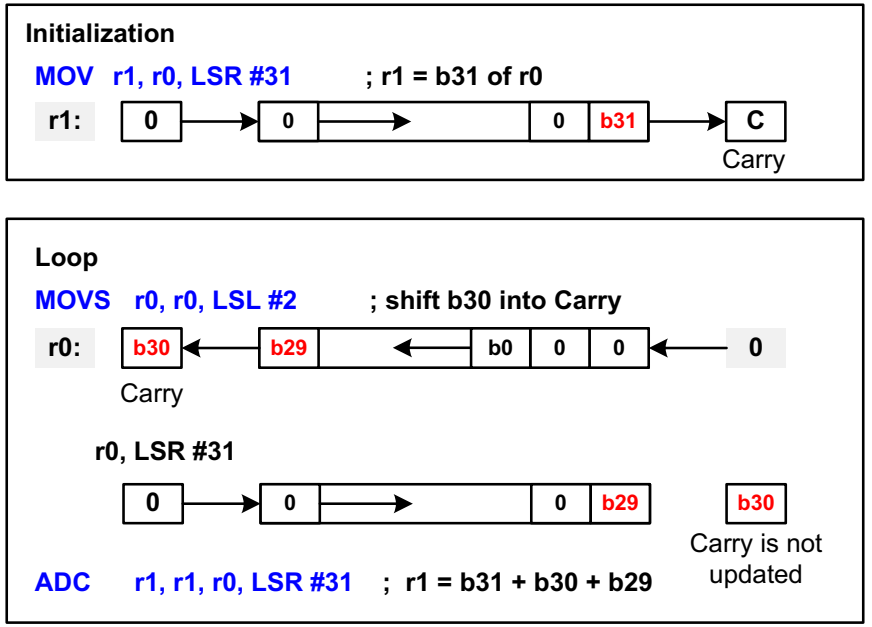
```
Assembly Program
        AREA Count_Ones, CODE
        EXPORT __main
        ALIGN
        ENTRY
__main  PROC
        ; r0 = Input = x
        ; r1 = Number of ones = counter
        LDR  r0, =0xAAAAAAAA

        ; r1 = r0 >> 31
        MOV  r1, r0, LSR #31

        ; r0 = r0 << 2 and change Carry
loop    MOVS r0, r0, LSL #2

        ; r1 = r1 + r0 >> 31 + Carry
        ADC    r1, r1, r0, LSR #31
        BNE  loop

 stop    B         stop
        ENDP
        END
```

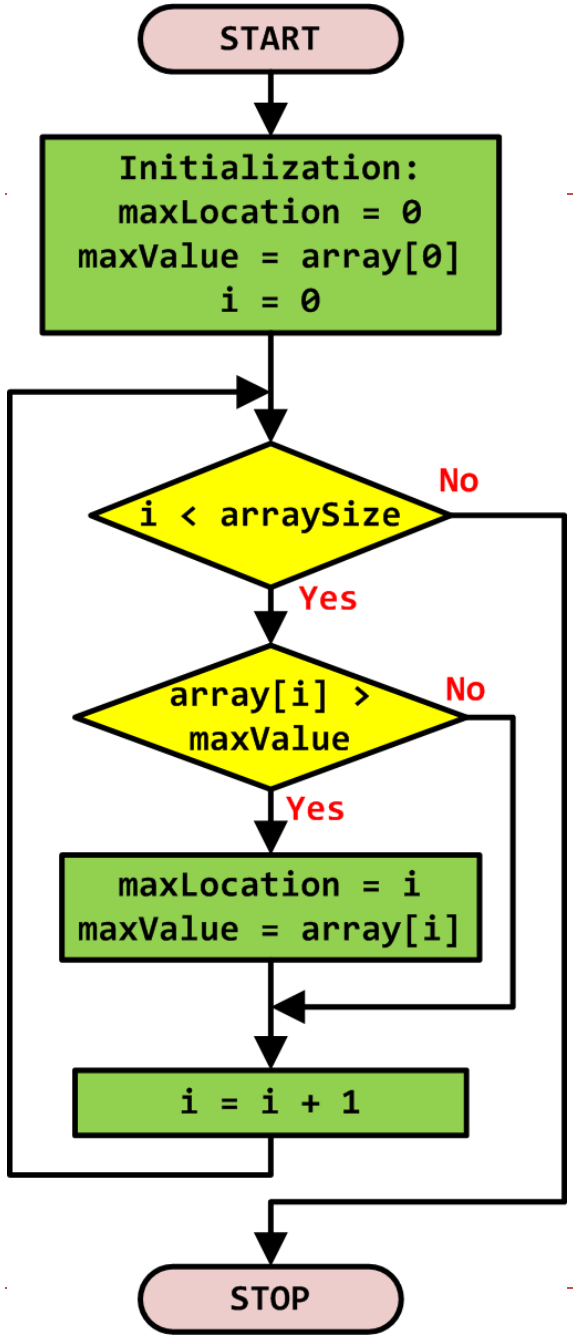



At the end of the first loop: **r1 = b31 + b30 + b29**

# Example 3: Finding Max of an Array

```
// Initialize max and location
maxLocation = 0;
maxValue = array[0];

// Loop through the array
for (i = 0; i < arraySize; i++) {
    if (array[i] > maxValue) {
        maxValue  = array[i];
        maxLocation = i;
    }
}
```



# Example 3: Finding Max of an Array

| C Program | Assembly Program |
|---|---|
| `int array[10] = {-1, 5, 3, 8, 10, 23, 6, 5, 2, -10};`<br><br>`int size = 10;` | `AREA myData, DATA`<br>`ALIGN`<br>`array DCD -1,5,3,8,10,23,6,5,2,-10`<br>`size DCD 10` |
| `int main(void) {`<br>`  int i, maxLocation, maxValue;` | `AREA findMax, CODE`<br>`EXPORT __main`<br>`ALIGN`<br>`ENTRY`<br>`__main PROC`<br>`; Identify the array size`<br>`LDR r3, =size`<br>`LDR r3, [r3] ; array size`<br>`SUB r3, r3, #1` |
| `  // Initialize max and location`<br>`  maxLocation = 0;`<br>`  maxValue = array[0];` | `; Initialize max value and location`<br>`LDR r4, =array`<br>`LDR r0, [r4] ; r0 = default max`<br>`MOV r1, #0 ; r1 = max location` |
| `  // Loop through the array`<br>`  for (i = 0; i < size; i++){`<br>`  if (array[i] > maxValue) {`<br>`  maxValue = array[i];`<br>`  maxLocation = i;`<br>`  }`<br>`  }` | `; Loop over the array`<br>`MOV r2, #0 ; loop index i`<br>`loop CMP r2, r3 ; compare i & size`<br>`BGE stop ; stop if i ≥ size`<br>`LDR r5, [r4,r2,LSL #2] ; array[i]`<br>`CMP r5, r0 ; compare with max`<br>`MOVGT r0, r5 ; update max value`<br>`MOVGT r1, r2 ; update location`<br>`ADD r2, r2, #1 ; update index i`<br>`B loop` |
| `  while(1); //dead loop`<br>`}` | `stop B stop ; dead loop`<br>`ENDP`<br>`END` |